

Aula 20: Introdução à UML, Revisão de OO e Casos de Uso

Fundamentação Teórica e Prática baseada nos Capítulos 1, 2 e 3 (Gilleanes Guedes)

Prof. Reginaldo Pereira Fernandes
Técnico em Informática para Internet

Instituto Federal do Ceará – Campus Tauá

28 de setembro de 2026

Sumário da Aula

- 1 Introdução à UML e Importância da Modelagem (Cap. 1)
- 2 Revisão de Orientação a Objetos (Cap. 2)
- 3 Diagrama de Casos de Uso (Cap. 3)
- 4 Atividade Prática e Exemplo de Modelagem

Por que Modelar Software? O Paradoxo da Construção

A Metáfora da Construção Civil:

- **Casinha de cachorro:** Qualquer pessoa constrói em poucas horas sem projeto. Se cair, o prejuízo é desprezível.
- **Casa de um andar:** Exige esboços básicos, mas desvios são toleráveis e corrigíveis.
- **Edifício de vários andares:** Jamais se inicia a obra sem plantas arquitetônica, hidráulica, elétrica e estrutural!

No Software Moderno:

- Sistemas corporativos e aplicações web comerciais operam como edifícios: envolvem regras de negócio, dados, segurança e alta concorrência.

Custo do Retrabalho

Corrigir uma falha estrutural de requisitos na fase de codificação ou em produção pode custar até **100 vezes mais** do que identificá-la na modelagem.

Fatores de Mudança Contínua

- Mudanças no mercado;
- Novas exigências tributárias e legais;
- Evolução contínua das demandas do cliente.

O Domínio do Problema:

- **Requisitos Funcionais (RF):** O que o sistema deve fazer (ex.: cadastrar cliente, emitir extrato, processar pedido).
- **Requisitos Não Funcionais (RNF):** Como o sistema deve operar (ex.: tempo de resposta, padrões de segurança, escalabilidade, regras de negócio).

Ruídos de Comunicação:

- O cliente relata necessidades usando termos vagos de negócio; o desenvolvedor raciocina com bancos de dados e código.

A Técnica da Prototipação (RAD)

Desenvolver telas e protótipos de baixa ou alta fidelidade valida requisitos com o usuário antes de codificar.

O que é um Modelo de Software?

É uma abstração simplificada que omite detalhes secundários para focar nos aspectos essenciais sob uma perspectiva específica.

Breve Histórico da UML: Da Fragmentação ao Padrão

A “Guerra dos Métodos” (Início dos Anos 90):

- Dezenas de metodologias concorrentes disputavam o mercado, gerando incompatibilidade de notações entre equipes.

Os “Três Amigos” (Three Amigos):

- **Grady Booch:** Criador do Método Booch;
- **James Rumbaugh:** Criador do Método OMT (*Object Modeling Technique*);
- **Ivar Jacobson:** Criador do Método OOSE (*Object-Oriented Software Engineering* - pioneiro dos Casos de Uso).

A Padronização pela OMG

- Reunidos na **Rational Software**, unificaram seus métodos no **Método Unificado** (1995);
- Em 1997, a **OMG** (*Object Management Group*) adotou a UML 1.1 como padrão global;
- Em 2005, lançamento da **UML 2.0**, trazendo forte reestruturação em 14 diagramas oficiais.

Por que a UML Possui Tantos Diagramas?

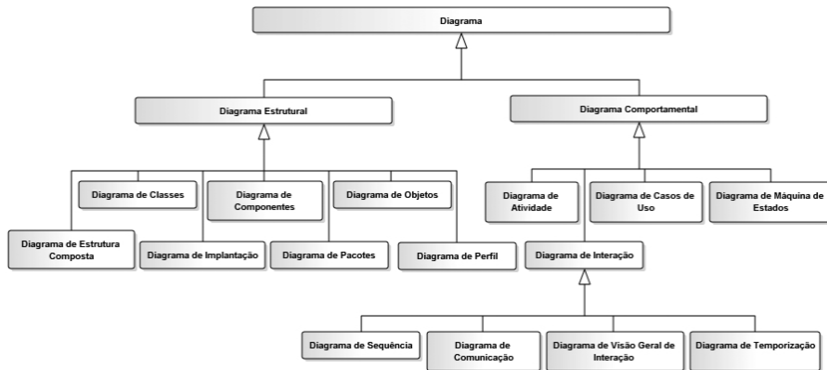
- **Múltiplas Visões Complementares:** Um software complexo não pode ser compreendido satisfatoriamente sob uma única perspectiva sem sobrecarregar a notação.
- **Públicos e Níveis de Abstração Distintos:**
 - **Clientes e Analistas de Negócio:** Necessitam de diagramas conceituais de alto nível (ex.: Casos de Uso, Atividades);
 - **Arquitetos e Desenvolvedores:** Necessitam de diagramas técnicos de projeto (ex.: Classes, Sequência, Componentes);
 - **Engenheiros de Infraestrutura/DevOps:** Necessitam de diagramas físicos de hardware e rede (ex.: Implantação).

Analogia com a Engenharia Civil

Assim como um edifício requer plantas distintas (arquitetônica, hidráulica, sanitária e elétrica), a UML fornece 14 diagramas especializados para cada dimensão do sistema.

Taxonomia Oficial dos 14 Diagramas da UML 2

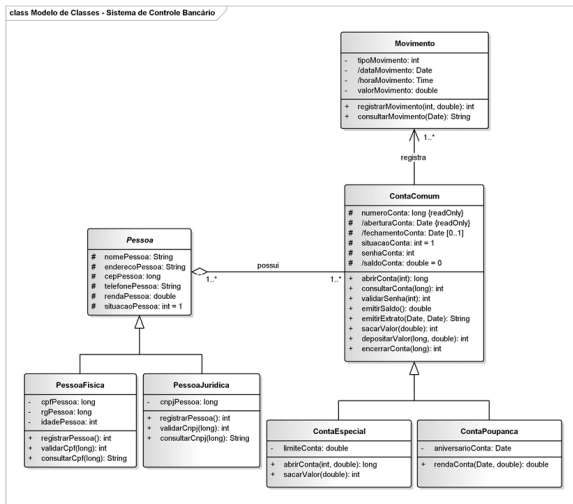
Organograma Geral da UML 2 (Figura 1.15 do Livro):



A UML divide-se em 7 Diagramas Estruturais e 7 Diagramas Comportamentais (incluindo 4 de Interação).

Diagrama de Classes: Visão Geral

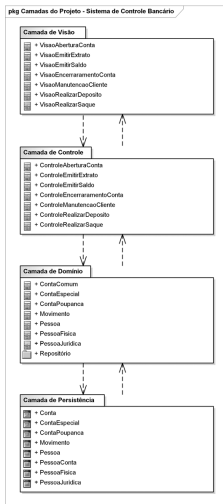
Exemplo de Diagrama de Classes (Figura 1.2 do Livro):



É o diagrama estrutural mais importante da UML, exibindo classes, atributos, métodos e associações estáticas.

Diagrama de Pacotes: Organização em Módulos

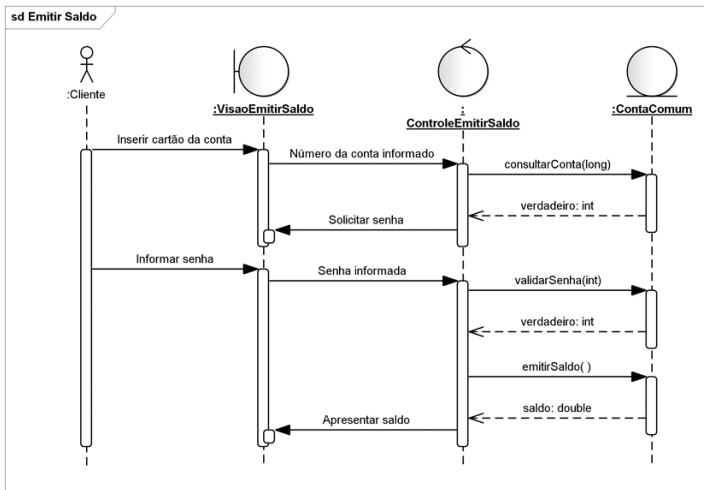
Exemplo de Diagrama de Pacotes (Figura 1.4 do Livro):



Agrupar elementos do modelo logicamente em subsistemas ou camadas arquiteturais (Apresentação, Negócio, Acesso a Dados).

Diagrama de Sequência: Interação Temporal

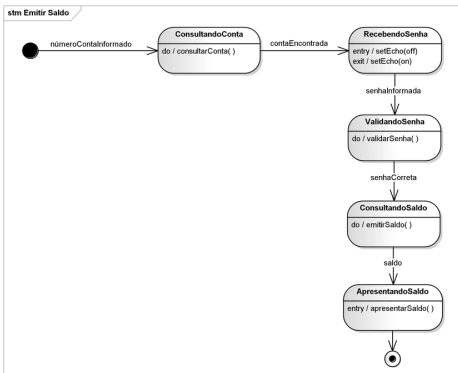
Exemplo de Diagrama de Sequência (Figura 1.5 do Livro):



Enfatiza a ordem cronológica em que mensagens e dados são trocados entre objetos ao longo de suas linhas de vida.

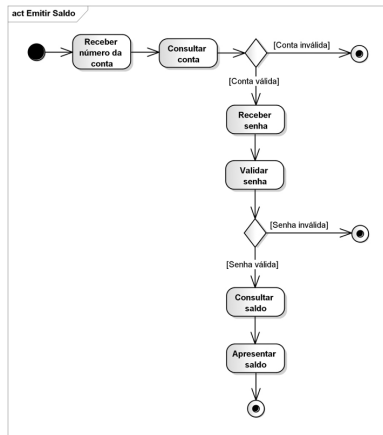
Máquina de Estados e Atividades (Página Única)

Máquina de Estados (Fig. 1.7):



Ciclo de vida e transições de estado de um objeto sob eventos.

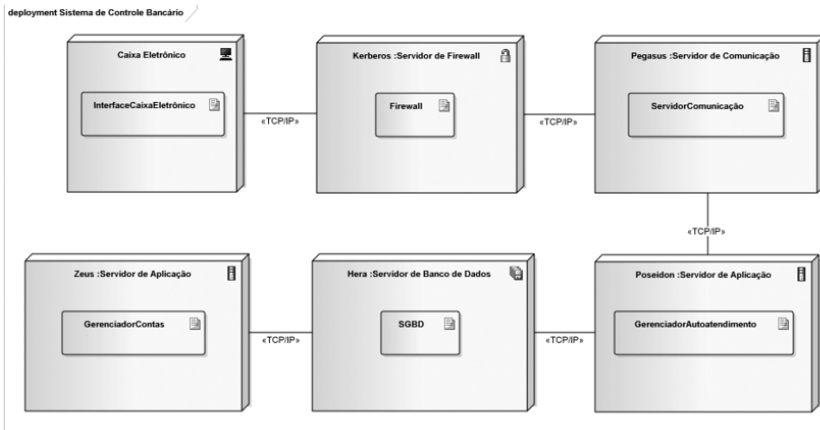
Atividade (Fig. 1.8):



Fluxo sequencial e paralelo de passos, decisões e algoritmos.

Diagrama de Implantação: Arquitetura Física

Exemplo de Diagrama de Implantação (Figura 1.11 do Livro):



Especifica o hardware, servidores de aplicação, bancos de dados, estações cliente e protocolos de comunicação física.

O Paradigma de Orientação a Objetos

Fundamentação Cognitiva:

- O pensamento humano categoriza e abstrai a realidade continuamente através de moldes conceituais (ex.: o conceito geral de “carro” ou “pessoa”).

Pilares Essenciais:

- **Abstração:** Isolar as características essenciais para o contexto do sistema, descartando detalhes irrelevantes.
- **Classificação:** Definir o modelo (molde) que agrupa características e comportamentos comuns.
- **Instanciação:** Materializar um objeto individual na memória com valores concretos em seus atributos.

Classe vs. Objeto

- **Classe:** Planta baixa, tipo de dado abstrato, definição estrutural.
- **Objeto:** Instância real em memória, com identidade, estado e comportamento.

Estado de um Objeto

O conjunto de valores armazenados em seus atributos em um dado instante define o seu estado.

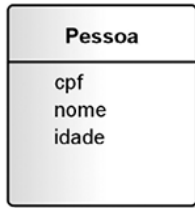
Anatomia de uma Classe na UML: 3 Divisões

Na UML, uma classe é representada por um retângulo com até **três divisões**:

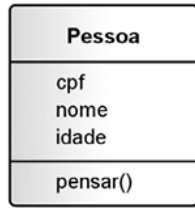
1. **Apenas Nome (2.1)**
2. **Com Atributos (2.2)**
3. **Com Métodos (2.3)**



Visão condensada: nome centralizado e destacado.



2ª divisão: propriedades que compõem o estado.



3ª divisão: operações e comportamentos executáveis.

Modificadores de Visibilidade e Tipos

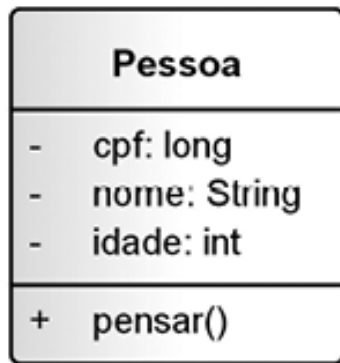
Notação de Visibilidade na UML:

- **Pública (+):** Acessível por qualquer classe externa;
- **Privada (-):** Acessível exclusivamente por métodos da própria classe;
- **Protegida (#):** Acessível pela própria classe e por suas subclasses derivadas;
- **Pacote (~):** Acessível por elementos pertencentes ao mesmo pacote.

Encapsulamento

Atributos costumam ser **privados** ou **protegidos**, enquanto operações públicas formam a interface de comunicação segura.

Classe Completa (Figura 2.4):



Herança: Generalização e Especialização

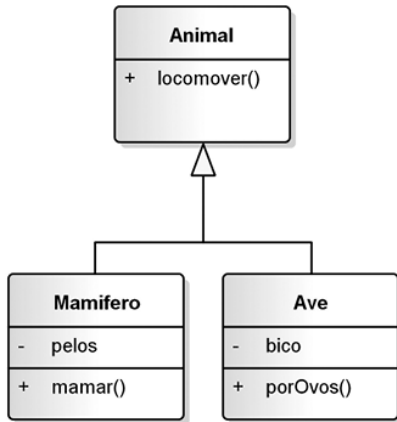
Mecanismo de Reuso:

- **Superclasse (Mãe/Base):** Define atributos e métodos genéricos compartilhados.
- **Subclasse (Filha/Derivada):** Herda os membros da superclasse e acrescenta características exclusivas.

Vantagens:

- Reutilização direta de código testado;
- Facilidade de manutenção centralizada;
- Extensibilidade sem duplicação de lógica.

Hierarquia Animal (Figura 2.5):



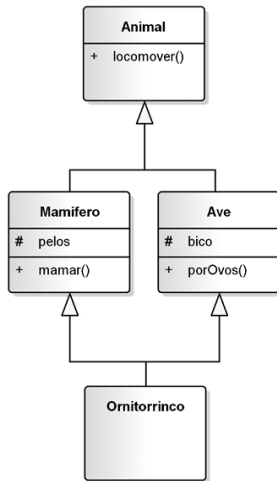
Herança Múltipla (Figura 2.6):

Conceito:

- Uma subclasse herda diretamente atributos e métodos de **duas ou mais** superclasses simultaneamente.

O Exemplo de Guedes (Figura 2.6):

- O Ornitorrinco herda pelos e capacidade de mamar da classe Mamífero;
- Herda bico e capacidade de pôr ovos da classe Ave;
- Herda a capacidade de locomoção da superclasse comum Animal.



Polimorfismo e Sobrescrita de Métodos

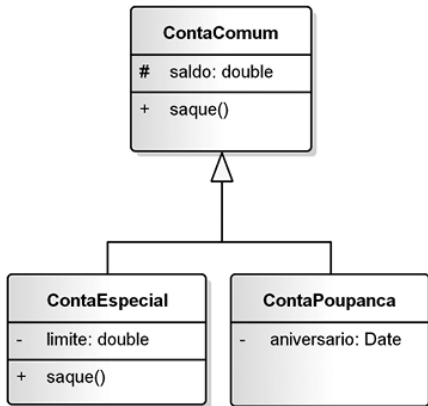
Conceito Central:

- “Muitas formas”: capacidade de objetos de subclasses distintas responderem à **mesma mensagem** com comportamentos especializados.

O Caso Bancário (Figura 2.7):

- ContaComum: possui #saldo e método +saque(), restrito ao saldo existente;
- ContaEspecial: adiciona #limite e sobrescreve (*override*) o método +saque() para permitir saque até a soma de saldo e limite.

Modelo Polimórfico (Figura 2.7):



Mapeamento Conceitual: Do Modelo ao Código

No arquivo `exemplo_poo.py`, o modelo da Figura 2.7 é implementado:

```
class ContaComum:
    def __init__(self, numero: str, saldo: float = 0.0):
        self._saldo = float(saldo) # Protegido (#)
    def saque(self, valor: float) -> bool:
        if 0 < valor <= self._saldo:
            self._saldo -= valor
            return True
        return False

class ContaEspecial(ContaComum):
    def __init__(self, numero: str, saldo: float = 0.0, limite: float =
        500.0):
        super().__init__(numero, saldo)
        self._limite = float(limite) # Protegido (#)
    def saque(self, valor: float) -> bool: # Sobrescrita Polimorfica
        if 0 < valor <= (self._saldo + self._limite):
            self._saldo -= valor # Saldo pode ficar negativo
            return True
        return False
```

O que é o Diagrama de Casos de Uso?

Finalidade Central:

- Criado originalmente por **Ivar Jacobson** em 1986 (metodologia OOSE).
- Apresenta uma visão externa de “**caixa-preta**” do sistema.
- Mapeia **quem** interage com o sistema e **quais serviços de valor** o sistema disponibiliza.

O que NÃO é Casos de Uso?

Não se preocupa com telas, bancos de dados, linguagens ou algoritmos internos. O foco reside nas funcionalidades do usuário!

Elemento de Ligação

O diagrama de casos de uso é a principal ponte de comunicação entre clientes e a equipe de desenvolvimento.

Elementos Centrais:

- 1 Atores;
- 2 Casos de Uso;
- 3 Associações;
- 4 Fronteira de Sistema.

Atores no Diagrama de Casos de Uso

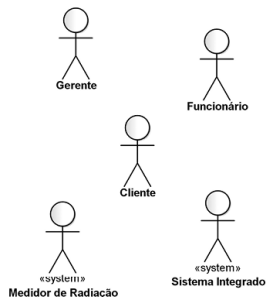
Definição de Ator:

- Representa um **papel** desempenhado por uma entidade externa que interage diretamente com o sistema.
- **Atores Humanos:** Clientes, Gerentes, Atendentes, Entregadores.
- **Atores Não Humanos (Sistemas Externos):** Gateways de pagamento, servidores de e-mail, APIs de frete, sensores.

Como Identificar os Atores?

- Quem fornecerá dados ao sistema?
- Quem consumirá as saídas e relatórios?
- Quais sistemas externos realizam trocas de informações?

Representação do Ator (3.1):



Boneco palito (*stickman*) com nome do papel abaixo.

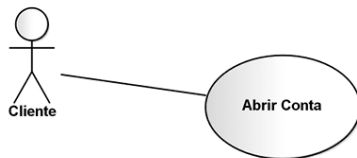
O que é um Caso de Uso? (3.2)

- Representa uma funcionalidade ou serviço completo entregue pelo sistema.
- Gráficamente é uma **elipse**.
- **Regra de Nomeação:** Verbo no infinitivo seguido de substantivo (ex.: *Consultar Saldo*, *Finalizar Pedido*).



Associação com Ator (3.3)

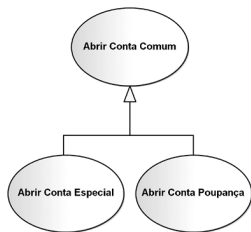
- Uma linha contínua conectando o ator ao caso de uso.
- Indica comunicação direta: o ator inicia o caso de uso ou recebe dados dele.



Generalização e Especialização

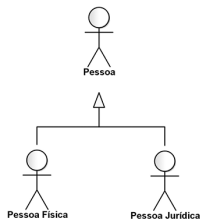
Entre Casos de Uso (3.4):

- O caso filho herda o comportamento do caso pai e especializa etapas específicas.
- Exemplo: *Abrir Conta Especial* herda de *Abrir Conta*.



Entre Atores (3.5):

- Um ator especializado herda todos os casos de uso do ator mais geral.
- Exemplo: *Cliente Especial* herda todos os direitos de *Cliente Comum*.

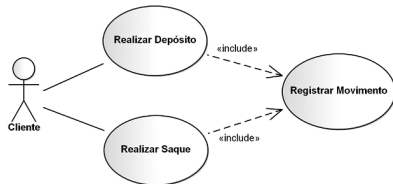


Relacionamento de Inclusão (<<include>>)

Conceito Central:

- Utilizado quando uma sequência de passos é **comum a múltiplos casos de uso**.
- A execução do caso de uso incluído é **OBRIGATÓRIA!**
- O caso base não se completa sem o término com sucesso do caso incluído.
- Evita redundância de especificação.

Exemplo do Livro (3.7):



Saque e Depósito incluem obrigatoriamente Registrar Movimento.

Direção da Seta no Include

A seta aponta do **Caso Base** para o **Caso**

Incluído: Caso Base $\xrightarrow{\text{include}}$ Caso Incluído

Relacionamento de Extensão (<<extend>>)

Conceito Central:

- Utilizado para modelar fluxos e cenários **OPCIONAIS** ou **CONDICIONAIS**.
- O caso de uso extensor só é acionado sob certas circunstâncias ou decisão do usuário.
- O caso base pode ser concluído normalmente sem a execução do caso estendido.

Exemplo com Restrição (3.12):



Autorregistrar estende Realizar Login caso o cliente não tenha cadastro.

Direção da Seta no Extend

A seta aponta do **Caso Extensor** (opcional) para o **Caso Base**:
Caso Extensor $\xrightarrow{\text{extend}}$ Caso Base

Pontos de Extensão (*Extension Points*)

O que é um Ponto de Extensão?

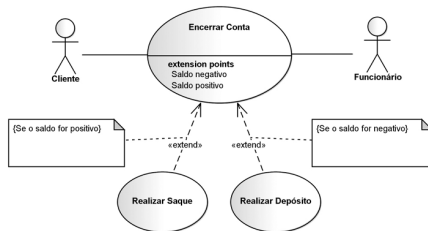
- Identifica exatamente **em qual etapa** do caso de uso base o comportamento opcional pode ser disparado.
- Na elipse da UML, cria-se um compartimento inferior nomeado com os pontos de extensão.

Caso do Livro de Guedes (3.14)

Ao *Encerrar Conta*, os pontos de extensão verificam:

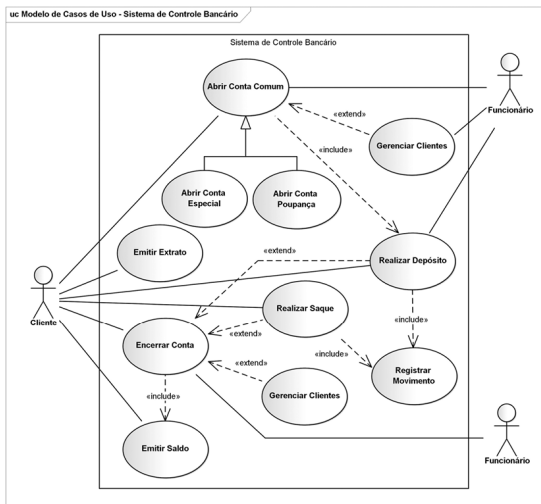
- Se saldo positivo: dispara *Realizar Saque*;
- Se saldo negativo: dispara *Cobrar Débito*.

Pontos de Extensão (Figura 3.14):



Fronteira do Sistema: Sistema Bancário Completo

Fronteira do Sistema e Casos de Uso (Figura 3.17 do Livro):



O retângulo delimita o software: Casos de Uso ficam DENTRO; Atores ficam FORA.

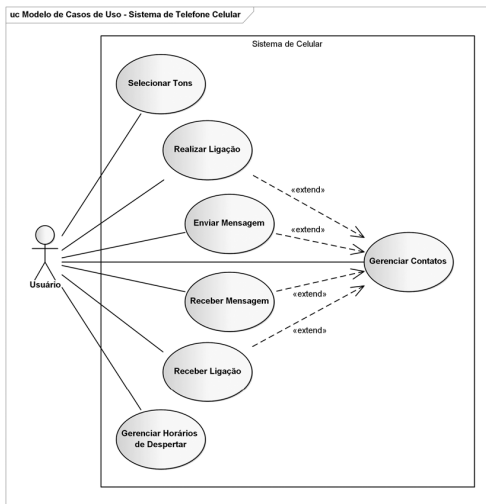
Gilleanes Guedes destaca: **o diagrama não é suficiente!** Ele precisa ser acompanhado da especificação detalhada de cenários:

Campos Padronizados da Ficha Técnica (Seção 3.4 do Livro)

- **Identificador e Nome:** Código único e verbo de ação (ex.: *UC03 - Finalizar Pedido*).
- **Atores Envolvidos:** Ator primário (iniciador) e secundários (sistemas de apoio).
- **Pré-condições:** Requisitos que devem ser verdadeiros antes do início do fluxo.
- **Pós-condições:** Estado garantido do sistema após execução com sucesso.
- **Fluxo Principal (Caminho Feliz):** Sequência ordenada de passos onde tudo ocorre com sucesso.
- **Fluxos Alternativos:** Caminhos secundários de navegação ou decisões opcionais do usuário.
- **Fluxos de Exceção:** Tratamento de erros, falhas técnicas e validações frustradas.

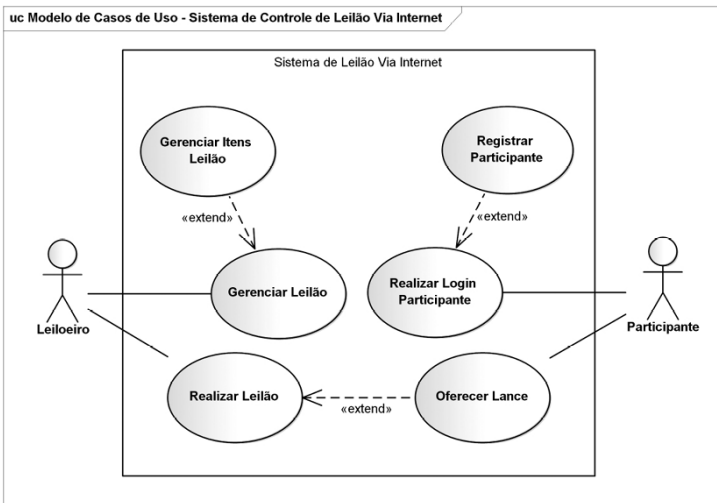
Estudo de Caso: Sistema de Telefonia Celular

Diagrama de Casos de Uso - Telefonia (Figura 3.18 do Livro):



Observe a divisão funcional entre serviços de voz, mensagens e utilitários e os múltiplos atores.

Diagrama para Sistema Web de Leilão (Figura 3.25 do Livro):



Atores Comprador e Vendedor, Casos de Uso de lances, arremate, avaliação e notificações.

Atividade Prática: Proposta e Requisitos

Cenário: Sistema *TauáDelivery Web*

Modelagem do Diagrama de Casos de Uso e documentação textual para delivery online.

Requisitos a Modelar (RF01 a RF13):

- **Atores:** Cliente, Entregador, Restaurante Parceiro e Gateway de Pagamento;
- **Casos de Uso:** Consultar Cardápio, Gerenciar Carrinho, Finalizar Pedido, Cancelar Pedido, Atualizar Status, etc.
- **Inclusões:** Finalizar Pedido inclui Autenticar e Processar Pagamento;
- **Extensões:** Finalizar Pedido estendido por Aplicar Cupom.

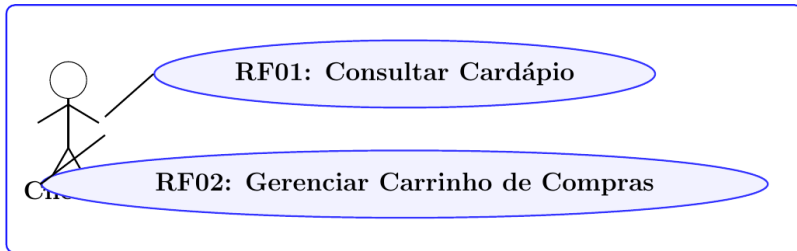
Entregáveis Solicitados:

- 1 **Parte 1:** Análise conceitual de atores e relacionamentos;
- 2 **Parte 2:** Diagrama de Casos de Uso completo com fronteira;
- 3 **Parte 3:** Especificação técnica formal de *UC03 - Finalizar Pedido* (Fluxos Principal, Alternativo e Exceção).

Exemplo de Modelagem: Requisitos RF01 e RF02

Diagrama de Casos de Uso para RF01 e RF02 (Modelo de Apoio):

Fronteira do Sistema: TauáDelivery Web



Comentário do Modelo de Referência

O ator **Cliente** conecta-se diretamente aos casos de uso **RF01: Consultar Cardápio** e **RF02: Gerenciar Carrinho de Compras**, ambos contidos dentro da **Fronteira do Sistema**.

Especificação de Apoio: Casos de Uso UC01 e UC02

Ficha UC01 - Consultar Cardápio

Ator: Cliente

Pré-condição: Sistema online com restaurantes cadastrados.

Pós-condição: Cardápio exibido com fotos e preços.

Fluxo Principal: 1. Cliente acessa página inicial;

2. Sistema lista restaurantes abertos;

3. Cliente escolhe restaurante;

4. Sistema exibe cardápio por categoria;

5. Cliente clica no item para detalhes.

Fluxo Alternativo: Busca por categoria ou prato.

Fluxo de Exceção: Restaurante fora do horário de funcionamento.

Ficha UC02 - Gerenciar Carrinho

Ator: Cliente

Pré-condição: Cliente visualizando cardápio (UC01).

Pós-condição: Itens e valores totais calculados na sessão.

Fluxo Principal: 1. Cliente clica em "Adicionar ao Carrinho";

2. Sistema valida disponibilidade e adiciona item;

3. Sistema atualiza contador e subtotal;

4. Cliente abre carrinho para conferência;

5. Sistema lista itens, quantidades e total.

Fluxo Alternativo: Alteração de quantidade ou remoção.

Fluxo de Exceção: Alerta de troca de restaurante no carrinho.

Diretriz para os Alunos

Utilizem essa mesma estrutura formal para redigir o caso de uso **UC03: Finalizar Pedido!**

Conclusão e Próximos Passos

- **A UML é a linguagem franca do software:** Permite pensar a arquitetura e alinhar requisitos antes de codificar.
- **A Orientação a Objetos é o alicerce:** Encapsulamento, herança e polimorfismo dão estrutura e flexibilidade ao domínio.
- **Casos de Uso definem o escopo do projeto:** Mapeiam de forma clara o que o software deve entregar do ponto de vista do usuário.

Próxima Aula (Aula 21)

Aprofundamento no Diagrama de Classes e Diagrama de Atividades:

Associações complexas (multiplicidades, navegabilidade, agregação e composição) e modelagem de fluxos procedurais.